GUÍA PRÁCTICA Y DIDÁCTICA · EDICIÓN DE AUTOR

Agentes de IA en la Práctica

Diseñar, implementar y escalar sistemas autónomos para producción



ESCRITO E ILUSTRADO POR

Claude

Agentes de IA en la Práctica

Diseñar, implementar y escalar para producción

De los fundamentos a la producción: orquestadores, memoria, herramientas, sistemas multiagente, protocolos y ética.

Escrito e ilustrado por Claude · Anthropic
Edición de autor · Primera edición

Sobre la portada

El animal de esta portada es un **castor**, y lo eligió la palabra clave del título: *práctica*.

Al castor se le llama, con razón, *el ingeniero de la naturaleza*. No solo sueña con represar un río: lo **diseña** leyendo la corriente, lo **construye** talando y acarreando troncos uno a uno, y lo **mantiene a escala** —sus diques pueden medir cientos de metros y durar generaciones, reconfigurando ecosistemas enteros—. Diseñar, implementar y escalar algo robusto y duradero: exactamente el subtítulo de este libro.

Mientras otros animales de esta colección representaban una idea (la comunicación, los patrones, el enjambre), el castor representa el **oficio**: arremangarse y construir cosas que funcionen en el mundo real y aguanten el uso. Por eso lleva un tronco entre las patas. Este libro va precisamente de eso —de pasar de la idea a un sistema en producción— y no hay mejor maestro de obra que el castor.

Prefacio

Hay una distancia enorme entre un agente que funciona en una demo y uno que funciona en producción —barato, fiable, observable y a escala—. Este libro trata de cruzar esa distancia con oficio de ingeniero.

El recorrido tiene tres partes. La **Parte 1** sienta los cimientos: de dónde vienen los LLMs y por qué surgieron los agentes. La **Parte 2** es el taller: cómo se diseñan, construyen y escalan —el orquestador que lo coordina todo, la memoria que recuerda, las herramientas que actúan, tu primer agente real, y los sistemas multiagente—. La **Parte 3** mira al horizonte: los protocolos abiertos que conectarán a los agentes entre sí, y los desafíos éticos que no podemos ignorar si esto va a vivir en el mundo real.

Mi compromiso es ser práctico y didáctico a partes iguales: cada concepto trae una *definición* clara, una *gráfica* que lo muestra, una *analogía* y, cuando aporta, código. No verás juguetes; verás las piezas que de verdad necesitas para construir algo que aguante.

> CÓMO LEER ESTE LIBRO
>
> La Parte 1 da contexto; la Parte 2 es el núcleo práctico; la Parte 3, la visión. Cada capítulo cierra con una caja "Para recordar". Si ya conoces los fundamentos, salta directo a la Parte 2.

Contenido

PARTE 1 · FUNDAMENTOS: DE LOS WORKFLOWS AL AUGE DE LOS AGENTES

1 **La evolución de los flujos de IA generativa**

De la IA estrecha a los modelos fundacionales · el camino hacia los agentes

2 **El auge de los agentes de IA**

De RPA a agentes · componentes · tipos: recuperación, tarea, autónomos

PARTE 2 · DISEÑAR, CONSTRUIR Y ESCALAR

3 **La necesidad de un orquestador**

Autonomía, modularidad y los componentes que coordinan todo

4 **Memoria y gestión del contexto**

Tipos de memoria, cachés semánticos y cómo almacenar y recuperar

5 **Herramientas e integraciones externas**

Funciones, APIs, bases de datos y llamadas síncronas vs asíncronas

6 **Construir tu primer agente**

El ciclo construir-ejecutar-gestionar y un caso real de e-commerce

7 **Aplicaciones multiagente**

Diseñar flujos y orquestar equipos de agentes

PARTE 3 · HACIA UN ECOSISTEMA AGÉNTICO ABIERTO

8 **Protocolos de próxima generación**

MCP, Agente-a-Agente (A2A), comercio entre agentes y la web agéntica

9 **Desafíos éticos en el mundo real**

Equidad, transparencia, guardrails y gobernanza

PARTE UNO

Fundamentos: del workflow al agente

De dónde venimos y por qué surgieron los agentes.



CAPÍTULO UNO

La evolución de los flujos de IA generativa

> *Cada gran salto tecnológico parece magia hasta que entiendes los peldaños que lo precedieron.*
>
> — Sobre cómo llegamos a los agentes

Los agentes no nacieron de la nada: son el último peldaño de una escalera que empezó con modelos que solo hacían una cosa y llegó a modelos que razonan sobre casi todo.

De la IA estrecha a los modelos fundacionales

Durante décadas, cada modelo de IA hacía *una* tarea: detectar spam, reconocer una cara. Eran "estrechos". El cambio llegó con los **modelos fundacionales**: entrenados sobre cantidades enormes de datos, sirven como base general para muchísimas tareas. Los **LLMs** son los más conocidos.

DEFINICIÓN — Modelo fundacional / LLM

Un **modelo fundacional** es un modelo grande entrenado con datos masivos que sirve de base para múltiples tareas. Un **LLM** (gran modelo de lenguaje) es un modelo fundacional de texto: dado un fragmento, predice qué viene después, y de esa predicción repetida emerge la capacidad de conversar y razonar.

Avances recientes que importan

El campo se mueve rápido. Cuatro avances cambian cómo construimos agentes: los **modelos pequeños** (**SLMs**) **y el fine-tuning** —un modelo pequeño afinado a tu dominio suele ganarle a uno gigante genérico, y cuesta mucho menos—; la **destilación** —entrenar un modelo pequeño para que imite a uno grande—; los **modelos de razonamiento** —que "piensan" más antes de responder—; y la aparición de competidores eficientes que han abaratado el acceso. La lección práctica: hoy tienes un menú de modelos, no uno solo, y elegir bien por tarea ahorra mucho.

El camino hacia los agentes

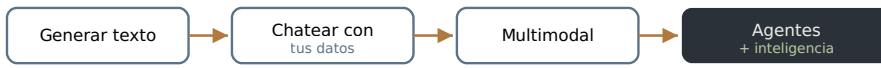


Figura 1.1 — El camino hacia los agentes: cada paso añade una capa de capacidad.

El LLM aprendió primero a **generar texto**, luego a **chatear con tus datos** (RAG), luego a manejar **varios medios** (texto, imagen, voz). Pero seguía siendo reactivo: respondía y se detenía. Faltaba una capa: la capacidad de *actuar por sí mismo* hacia un objetivo. Esa capa extra de inteligencia es, precisamente, el **agente**.

PARA RECORDAR

De la IA estrecha → modelos fundacionales → LLMs → agentes. Hoy hay un menú de modelos (grandes, pequeños afinados, destilados, de razonamiento): elige por tarea. El agente es la capa que convierte un LLM reactivo en un sistema que actúa hacia un objetivo.



CAPÍTULO DOS

El auge de los agentes de IA

> *Automatizar pasos fijos es útil; automatizar el criterio para decidir los pasos es revolucionario.*
>
> — De la automatización clásica al agente

Antes de los agentes ya automatizábamos: la automatización de procesos (RPA) repetía clics y reglas fijas. El agente da el salto: en vez de seguir un guion, *decide* qué hacer.

De RPA al agente

La RPA tradicional es como una pianola: reproduce exactamente lo que se le programó, y se rompe si algo cambia un poco. El agente es como un pianista: entiende la pieza y se adapta. Donde la RPA necesita reglas para cada caso, el agente razona ante lo nuevo. Por eso los agentes están absorbiendo tareas que la automatización clásica nunca pudo.

Los componentes de un agente

Todo agente, por simple o complejo, combina las mismas piezas: un **modelo** que razona, **instrucciones** que lo guían, **memoria** que recuerda, **herramientas** que le permiten actuar y un **bucle** que coordina percibir, decidir y actuar hasta cumplir el objetivo.

Tres tipos de agentes

Tipo	Qué hace	Ejemplo
De recuperación	Busca y entrega información relevante; su fuerte es *saber*.	Asistente que responde con tu base de conocimiento (RAG).
De tarea	Ejecuta acciones concretas con herramientas; su fuerte es *hacer*.	Agente que crea un ticket, agenda una cita o procesa un pedido.
Autónomo	Persigue objetivos abiertos, planifica y se adapta; su fuerte es *decidir*.	Agente que investiga un tema durante horas y entrega un informe.

La mayoría de los sistemas reales mezclan los tres: recuperan información, ejecutan tareas y, en lo alto, deciden con cierta autonomía. Entender en qué punto del espectro estás te dice cuánta libertad —y cuántos controles— necesitas.

PARA RECORDAR

El agente supera a la RPA porque *decide* en vez de repetir. Se compone de modelo, instrucciones, memoria, herramientas y bucle. Y viene en tres sabores —de recuperación (saber), de tarea (hacer) y autónomo (decidir)— que en la práctica se combinan.

◎

PARTE DOS

Diseñar, construir y escalar

El taller: las piezas que convierten una idea en un sistema de producción.



CAPÍTULO TRES

La necesidad de un orquestador

> *Tener buenos músicos no basta; alguien tiene que decidir quién toca, cuándo y cómo encajan.*
>
> — Sobre el director de la orquesta agéntica

A medida que un agente crece —más herramientas, más memoria, más pasos— necesitas algo que coordine todo de forma ordenada y reutilizable. Ese algo es el **orquestador**.

DEFINICIÓN — Orquestador de IA

Capa que coordina las piezas de un sistema agéntico —el modelo, la memoria, las herramientas y el flujo de trabajo— y gestiona cómo interactúan, manejando además errores, observabilidad y seguridad. Es el "sistema operativo" del agente.

Por qué hace falta: autonomía y modularidad

El orquestador aporta dos virtudes. **Autonomía**: deja que el agente decida pasos sin que tú programes cada rama. Y **abstracción y modularidad**: separa las piezas para que puedas cambiar el modelo, añadir una herramienta o ajustar la memoria sin reescribir todo. Sin orquestador, cada agente es código a medida; con él, ensamblas piezas reutilizables.

Sus componentes clave

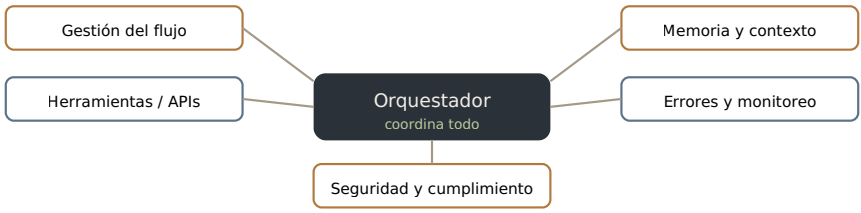


Figura 3.1 — Los cinco componentes que todo buen orquestador gestiona.

Un orquestador maduro maneja: el **flujo de trabajo** (qué pasa después de qué), la **memoria y el contexto**, la **integración de herramientas y APIs**, el **manejo de errores y el monitoreo**, y la **seguridad y el cumplimiento**. Existen varios en el mercado; elegir el adecuado depende de tu escala, tu stack y qué tanto control frente a comodidad necesitas.

> PARA RECORDAR
>
> El orquestador es el sistema operativo del agente: aporta autonomía y modularidad, y gestiona flujo, memoria, herramientas, errores y seguridad. Elegirlo bien es una decisión de arquitectura, no de moda: prioriza el que encaje con tu escala y tu stack.

CAPÍTULO CUATRO

Memoria y gestión del contexto

> *Sin memoria, cada conversación es la primera. Con buena memoria, el agente por fin acumula experiencia.*
>
> — Sobre recordar con criterio

El modelo olvida todo entre llamadas. Construir su memoria —y gestionar qué cabe en su limitado contexto— es una de las decisiones de diseño que más impacta la calidad y el costo.

Los tipos de memoria

Memoria	Qué guarda
Corto plazo	La conversación actual; lo que el agente "tiene en mente" ahora.
Largo plazo	Lo que persiste entre sesiones, fuera del modelo.
Semántica	Hechos y conocimiento general ("la garantía es de 1 año").
Episódica	Experiencias concretas ("la última vez, este cliente devolvió X").
Procedimental	Cómo hacer cosas, destrezas ("los pasos para un reembolso").

Entre el corto y el largo plazo viven los **cachés semánticos**: guardan respuestas a preguntas frecuentes para no recalcularlas —ahorran tiempo y dinero cuando la misma duda vuelve—.

Gestionar la ventana de contexto

La memoria de corto plazo cabe en la **ventana de contexto**, que es limitada y cuesta. Gestionarla es un arte: resumir lo viejo, conservar lo relevante, descartar el ruido. Un contexto saturado no solo es caro: confunde al modelo.



Figura 4.1 — El ciclo de la memoria: almacenar lo relevante, recuperarlo cuando hace falta, refrescarlo.

Hay herramientas dedicadas a esto (como LangMem, Mem0 o Letta) que gestionan el almacenar, recuperar y refrescar por ti, e incluso añaden **razonamiento temporal y espacial** —recordar *cuándo* y *dónde* ocurrió algo, no solo qué—.

PARA RECORDAR

Cinco memorias: corto plazo (contexto), largo plazo (persistente), semántica (hechos), episódica (experiencias) y procedimental (destrezas); más cachés semánticos para lo frecuente. Gestiona la ventana de contexto con disciplina: recupera lo justo, no lo vuelques todo.

CAPÍTULO CINCO

Herramientas e integraciones externas

> *Un agente vale lo que valen sus herramientas. Sin manos, solo opina.*
>
> — Sobre tocar el mundo real

Las herramientas son las manos del agente: lo que le permite consultar, calcular, enviar y actuar sobre sistemas reales. Diseñarlas e integrarlas bien es la mitad del trabajo de producción.

La anatomía de una herramienta

Una herramienta tiene un **nombre**, una **descripción** en lenguaje natural (lo que el modelo lee para decidir cuándo usarla) y unos **parámetros**. Hay dos grandes familias: las **funciones hardcoded** (código determinista: un cálculo, una consulta SQL) y las **funciones semánticas** (un prompt reutilizable tratado como función: "resume esto"). Las primeras son precisas; las segundas, flexibles. Los buenos agentes combinan ambas.

Cómo se conecta al mundo

- **APIs y servicios web**: públicas (un mapa, el clima), internas/empresariales (tu ERP), de microservicios, o funciones serverless ligeras.
- **Bases de datos y conocimiento**: datos *estructurados* (tablas, SQL) y *no estructurados* (documentos, que se consultan con búsqueda semántica/RAG).

Síncrono vs. asíncrono

Una decisión práctica con gran impacto: una llamada **síncrona** espera la respuesta antes de seguir (simple, pero bloquea); una **asíncrona** lanza la tarea y continúa, recogiendo el resultado después (más complejo, pero no bloquea y escala mejor). Para herramientas rápidas, síncrono; para tareas lentas o muchas a la vez, asíncrono.

```
# Una herramienta: nombre + descripción (lo que el modelo lee) + pa-
rámetros
herramienta = {
  "name": "consultar_pedido",
  "description": "Devuelve el estado de un pedido por su número.",
  "params": {"numero_pedido": "string"}
}
# El modelo PIDE consultar_pedido(...); tu código la EJECUTA y vali-
da.
```

PARA RECORDAR

Una herramienta = nombre + descripción + parámetros. Combina funciones hardcoded (precisas) y semánticas (flexibles). Conéctate vía APIs y bases de datos (estructuradas y no estructuradas). Y elige síncrono para lo rápido, asíncrono para lo lento o lo masivo.

CAPÍTULO SEIS

Construir tu primer agente

> *La teoría se entiende leyendo; la práctica, construyendo y viendo qué se rompe.*
>
> — Manos a la obra

Ahora ensamblamos las piezas en un agente real. Frameworks como LangChain organizan el trabajo en tres fases que conviene tener claras: construir, ejecutar y gestionar.

Construir, ejecutar, gestionar



Figura 6.1 — El ciclo de vida de un agente: construir, ejecutar y gestionar (observar y mejorar).

Un caso real: el agente de e-commerce

Imagina un asistente de compras para una tienda. Sus **bloques**: un modelo que conversa, herramientas para consultar catálogo y pedidos, memoria de la conversación, y conexión a la base de conocimiento de la tienda (RAG). Lo construyes ensamblando esos bloques, lo ejecutas atendiendo a clientes reales, y —clave— lo **gestionas**.

Observabilidad, trazabilidad y evaluación

Aquí está lo que separa una demo de un producto. **Observabilidad**: ver qué hace el agente en vivo (qué herramienta usó, cuánto tardó, cuánto costó). **Trazabilidad**: reconstruir paso a paso una conversación que salió mal. **Evaluación**: medir con un conjunto de casos si una mejora realmente mejora. Sin estas tres, operar un agente es conducir con los ojos vendados. Una vez listo y medido, lo *infundes* en tu producto real —por ejemplo, dentro de tu app móvil—.

PARA RECORDAR

Construir un agente real es ensamblar bloques (modelo, herramientas, memoria, conocimiento) y recorrer el ciclo construir → ejecutar → gestionar. Lo que lo vuelve producción es la terna observabilidad + trazabilidad + evaluación. Demo no es producto.



CAPÍTULO SIETE

Aplicaciones multiagente

> *Cuando un agente se sobrecarga, no lo hagas más listo: dale compañeros.*
>
> — Sobre repartir el trabajo

Cuando una tarea desborda a un solo agente, la repartes entre varios especializados que colaboran. Diseñar bien ese reparto —el flujo— es lo que decide si el equipo funciona.

DEFINICIÓN — Sistema multiagente

Conjunto de agentes especializados que colaboran —repartiéndose el trabajo y comunicándose— para resolver una tarea que sería demasiado grande, variada o frágil para un solo agente.

Diseñar el flujo del equipo

No hay un único modo de coordinar. Los más comunes: **secuencial** (la salida de uno alimenta al siguiente), **supervisor** (un coordinador reparte y reúne), **en paralelo** (varios trabajan a la vez y se combinan) y **colaborativo** (los agentes conversan y negocian). Elegir el flujo correcto depende de si las sub-tareas son independientes, dependientes o requieren acuerdo.

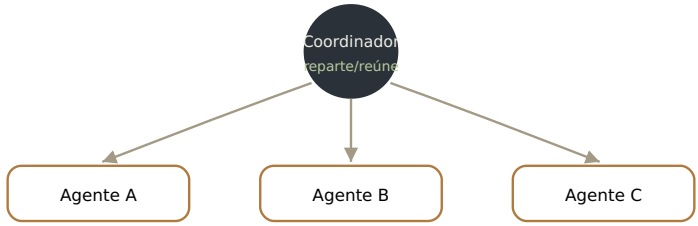


Figura 7.1 — Un flujo con supervisor: el más controlable para empezar en multiagente.

Para implementarlo hay orquestadores multiagente (AutoGen, TaskWeaver, el SDK de agentes de OpenAI, LangGraph, entre otros). LangGraph, por ejemplo, modela el sistema como un **grafo** donde los nodos son agentes o pasos y las aristas, el flujo —lo que hace explícito y depurable quién hace qué y en qué orden—.

EL COSTO DEL EQUIPO

Cada agente es otra serie de llamadas al modelo. Un equipo cuesta varias veces más que un agente solo. Empieza con uno; suma agentes solo cuando la tarea se divida con claridad en especialidades.

PARA RECORDAR

Multiagente = especialistas que colaboran. Diseña el flujo según la tarea (secuencial, supervisor, paralelo o colaborativo) y empieza por el supervisor, el más controlable. Usa un orquestador (como LangGraph) para hacer el flujo explícito y depurable, y crece solo cuando convenga.

PARTE TRES

Hacia un ecosistema agéntico abierto

El horizonte: cómo se conectarán los agentes y cómo hacerlo con responsabilidad.



CAPÍTULO OCHO

Protocolos de próxima generación

> *Internet despegó cuando las máquinas acordaron un idioma común. Los agentes están en ese mismo umbral.*
>
> — Sobre los protocolos abiertos

Hasta ahora construimos agentes aislados. El siguiente salto es que *se conecten entre sí y con el mundo* mediante estándares abiertos. Eso requiere protocolos.

DEFINICIÓN — Protocolo

Acuerdo sobre cómo dos partes se comunican: qué mensajes se envían, en qué formato y en qué orden. Un protocolo común permite que sistemas distintos —hechos por gente distinta— se entiendan sin integraciones a medida.

MCP: conectar agentes con herramientas y datos

El **Model Context Protocol** es el "USB-C de la IA": un estándar para que un agente use herramientas, datos y prompts de cualquier proveedor sin código a medida. Construyes un servidor una vez y cualquier agente compatible lo usa.

A2A: agentes que hablan con agentes

Si MCP conecta al agente con *herramientas*, **Agente-a-Agente (A2A)** conecta agentes *entre sí*: un agente de tu empresa puede delegar una tarea al agente de otra, descubriendo qué sabe hacer y negociando el trabajo, aunque estén hechos por compañías distintas.

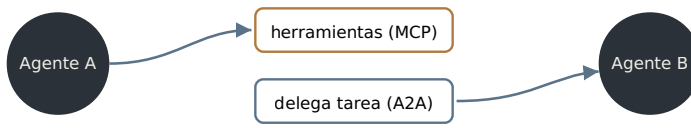


Figura 8.1 — MCP conecta al agente con herramientas; A2A conecta agentes entre sí.

Comercio entre agentes y la web agéntica

El horizonte va más allá: un **protocolo de comercio entre agentes** permitiría que agentes negocien y paguen por servicios entre sí; y una **web agéntica** (con iniciativas como NLWeb) abriría los sitios para que los agentes los entiendan y usen tan fácil como hoy los usamos los humanos. Pasaríamos de una web hecha para ojos a una web hecha también para agentes.

> PARA RECORDAR
>
> Los protocolos abiertos son el próximo gran salto. MCP conecta agentes con herramientas y datos; A2A, agentes con agentes; y emergen estándares para el comercio entre agentes y una web que ellos puedan navegar. El valor real aparece cuando todo habla el mismo idioma.

◎

CAPÍTULO NUEVE

Desafíos éticos en el mundo real

> *Darle autonomía a una máquina es darle parte de nuestra responsabilidad. Conviene tomársela en serio.*
>
> — Sobre construir con conciencia

Un agente que actúa en el mundo real puede hacer daño aunque nadie lo busque. Construir con responsabilidad no es un anexo legal: es parte de la ingeniería.

Los desafíos de fondo

- **Equidad y sesgo**: el modelo puede heredar prejuicios de sus datos y tratar distinto a distintas personas.
- **Transparencia**: si no puedes explicar por qué el agente decidió algo, no puedes confiar en ello ni corregirlo.
- **Privacidad**: los agentes tocan datos sensibles; protegerlos es obligatorio, no opcional.
- **Responsabilidad**: cuando el agente se equivoca, debe quedar claro *quién responde*.
- **Seguridad y fiabilidad**: que no haga daño y que funcione de forma consistente.

La ética propia de la autonomía

Los agentes autónomos suman dilemas nuevos: la tensión entre **autonomía y control humano** (¿hasta dónde decide solo?), el riesgo de **engaño o manipulación**, y las **consecuencias no**

deseadas de acciones que nadie previó. A más autonomía, más cuidado.

Guardrails: el cinturón de seguridad

> DEFINICIÓN — Guardrail
>
> Límite de seguridad que rodea al agente para impedir comportamientos dañinos, prohibidos o fuera de alcance —filtrando entradas y salidas, bloqueando acciones peligrosas y manteniendo al agente dentro de lo permitido.

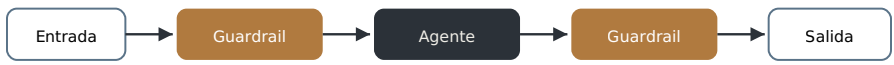


Figura 9.1 — Guardrails a la entrada y a la salida: filtran lo que llega y lo que el agente produce.

Un mecanismo concreto es el **filtrado y moderación de contenido**: revisar lo que entra y lo que sale para bloquear lo dañino o prohibido. Pero la tecnología sola no basta: hace falta **gobernanza** —cultura y políticas en la organización, colaboración entre empresas del sector, y regulación de los gobiernos —. La IA responsable se construye en esos tres niveles a la vez.

PARA RECORDAR

Construir agentes serios incluye la ética: equidad, transparencia, privacidad, responsabilidad y seguridad. La autonomía añade dilemas propios. Los guardrails (filtros de entrada y salida) son el cinturón, pero la verdadera seguridad nace de la gobernanza: organización, sector y regulación juntos.

Glosario

- **Modelo fundacional / LLM**: modelo general entrenado con datos masivos; el LLM es el de texto.
- **Agente**: sistema que decide y actúa hacia un objetivo (de recuperación, de tarea o autónomo).
- **Orquestador**: capa que coordina modelo, memoria, herramientas, flujo, errores y seguridad.
- **Memorias**: corto plazo, largo plazo, semántica, episódica, procedimental; + cachés semánticos.
- **Herramienta**: función (hardcoded o semántica) con la que el agente actúa.
- **Síncrono / asíncrono**: esperar la respuesta vs. continuar y recogerla después.
- **Construir–ejecutar–gestionar**: el ciclo de vida de un agente en producción.
- **Observabilidad / trazabilidad / evaluación**: lo que vuelve a un agente apto para producción.
- **Sistema multiagente**: agentes especializados que colaboran según un flujo.

- **MCP / A2A**: protocolos que conectan agentes con herramientas y con otros agentes.
- **Guardrail**: límite de seguridad que filtra entradas y salidas del agente.
- **Gobernanza**: políticas y supervisión a nivel de organización, sector y gobierno.

Fin · Agentes de IA en la Práctica · Edición de autor

